Machine Learning: Unit 2 (Detailed Notes)

Supervised Learning: Regression and Classification

Contents: 1. Intro to Supervised Learning · 2. Linear Regression · 3. Regression Metrics · 4. Logistic Regression · 5. Classification Intro · 6. k-NN · 7. Classification Metrics · 8. Practice

---

1. Introduction to Supervised Learning

1.1 Definition

In **supervised learning**, the algorithm learns a function **f: X → y** from a dataset of **labeled** examples (x_i, y_i). Afterwards it predicts y for unseen x. The labels act as a "supervisor".

```
Labeled data (X, y) → Learning algorithm → Model f
New data X_new → Model f → Prediction ŷ
```

1.2 Two Types of Problems

	Regression	Classification
Output	Continuous number	Discrete class
Examples	House price, salary, temperature	Spam/ham, disease yes/no, digit 0–9
Algorithms	Linear/Polynomial/Ridge/Lasso Regression, SVR	Logistic Regression, k-NN, Decision Tree, SVM, Naive Bayes
Metrics	MAE, MSE, RMSE, R^2	Accuracy, Precision, Recall, F1, AUC

1.3 Key Concepts

- **Training set** (learn) vs **Test set** (evaluate on unseen data).
- **Generalization:** performing well on new data.
- **Overfitting:** model memorizes noise → low train error, high test error (high variance).
- **Underfitting:** model too simple → high error on both (high bias).
- **Loss/Cost function:** measures error the model tries to minimize.
- **Gradient descent:** iterative optimizer: $\theta \leftarrow \theta - \alpha \cdot \partial J/\partial \theta$ (α = learning rate).

1.4 Standard Supervised Workflow in sklearn

```
import pandas as pd, numpy as np
from sklearn.model_selection import train_test_split

X = df.drop("target", axis=1)        # features
y = df["target"]                     # label

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)       # add stratify=y for classification
```

```
model = SomeModel()                 # 1. create
model.fit(X_train, y_train)         # 2. train
y_pred = model.predict(X_test)      # 3. predict
# 4. evaluate with metrics
```

`random_state` makes the split reproducible. `stratify=y` preserves class proportions.

---

2. Simple & Multiple Linear Regression

2.1 Simple Linear Regression (SLR)

Models a **linear relationship** between **one** independent variable x and a continuous dependent variable y.

Equation: $\hat{y} = \beta_0 + \beta_1 x$ (also written $y = mx + c$)

- **β_0 (intercept):** value of $\hat{y}$ when $x = 0$.
- **β_1 (slope):** change in $\hat{y}$ for one unit increase in x.
- **Residual (error):** $e_i = y_i - \hat{y}_i$.

Least Squares Method

Choose β_0, β_1 to minimize **SSE = $\Sigma(y_i - \hat{y}_i)^2$**.

$\beta_1 = \Sigma(x_i - \bar{x})(y_i - \bar{y}) / \Sigma(x_i - \bar{x})^2$ $\beta_0 = \bar{y} - \beta_1 \cdot \bar{x}$

Worked Numerical Example

Hours studied (x) vs Marks (y):

x	y	$x - \bar{x}$	$y - \bar{y}$	$(x-\bar{x})(y-\bar{y})$	$(x-\bar{x})^2$
1	2	−2	−2	4	4
2	4	−1	0	0	1
3	5	0	1	0	0
4	4	1	0	0	1
5	5	2	1	2	4

$\bar{x} = 3$, $\bar{y} = 4$ → $\Sigma(x-\bar{x})(y-\bar{y}) = 6$, $\Sigma(x-\bar{x})^2 = 10$ $\beta_1 = 6/10 =$ **0.6**, $\beta_0 = 4 - 0.6 \times 3 =$ **2.2** → **$\hat{y} = 2.2 + 0.6x$**. Prediction for $x = 6$ → $\hat{y} = 2.2 + 3.6 =$ **5.8**.

Code: Simple Linear Regression

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

# Sample data
df = pd.DataFrame({"hours": [1,2,3,4,5,6,7,8,9,10],
                   "marks": [35,40,50,55,60,65,72,78,85,90]})
```

```
X = df[["hours"]]            # 2-D (DataFrame) required by sklearn
y = df["marks"]              # 1-D

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)

model = LinearRegression()
model.fit(X_train, y_train)

print("Intercept (β0):", model.intercept_)
print("Slope (β1):", model.coef_[0])

y_pred = model.predict(X_test)
print("R2:", r2_score(y_test, y_pred))
print("Predict for 7.5 hrs:", model.predict(pd.DataFrame({"hours": [7.5]})))

# Visualize
plt.scatter(X, y, color="blue", label="Data")
plt.plot(X, model.predict(X), color="red", label="Regression line")
plt.xlabel("Hours studied"); plt.ylabel("Marks"); plt.legend(); plt.show()
```

2.2 Multiple Linear Regression (MLR)

Uses **two or more** features.

Equation: $\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$ **Matrix form (Normal Equation):** $\beta = (X^T X)^{-1} X^T y$ Each coefficient β_j = effect of x_j on $\hat{y}$ **holding other features constant**.

Example: Price = 50,000 + 3,000×(area) + 10,000×(bedrooms) − 500×(age of house).

Assumptions of Linear Regression (LINE + no multicollinearity)

1. **L**inearity: relation between X and y is linear.
2. **I**ndependence of errors (no autocorrelation).
3. **N**ormality of residuals.
4. **E**qual variance (homoscedasticity) of residuals.
5. **No multicollinearity** among features (check VIF < 5–10).

Code: Multiple Linear Regression

```
from sklearn.datasets import fetch_california_housing
from sklearn.preprocessing import StandardScaler

data = fetch_california_housing(as_frame=True)
df = data.frame
X = df.drop("MedHouseVal", axis=1)
y = df["MedHouseVal"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

mlr = LinearRegression().fit(X_train, y_train)

coef = pd.Series(mlr.coef_, index=X.columns)
print(coef.sort_values())
print("Intercept:", mlr.intercept_)

y_pred = mlr.predict(X_test)
```

```
print("R2  :", r2_score(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))

# Residual plot (should look random around 0)
plt.scatter(y_pred, y_test - y_pred, alpha=0.3)
plt.axhline(0, color="red"); plt.xlabel("Predicted"); plt.ylabel("Residual"); plt.show()

# Multicollinearity check
from statsmodels.stats.outliers_influence import variance_inflation_factor
vif = [variance_inflation_factor(X.values, i) for i in range(X.shape[1])]
print(dict(zip(X.columns, np.round(vif, 2))))
```

Handling categorical features in MLR

Convert with one-hot encoding first: `pd.get_dummies(df, columns=["city"], drop_first=True)` (avoids the dummy variable trap).

2.3 Gradient Descent (How models can be fitted)

Cost function $J(\beta) = (1/2n)\Sigma(\hat{y}_i - y_i)^2$. Repeat: $\beta \leftarrow \beta - \alpha \cdot \partial J/\partial \beta$ until convergence.

- α too small → slow; α too large → overshoots/diverges.

```
# Linear regression from scratch with gradient descent
X = np.array([1,2,3,4,5], dtype=float); y = np.array([2,4,5,4,5], dtype=float)
b0 = b1 = 0.0; lr = 0.01; n = len(X)
for _ in range(5000):
    y_hat = b0 + b1 * X
    b0 -= lr * (1/n) * np.sum(y_hat - y)
    b1 -= lr * (1/n) * np.sum((y_hat - y) * X)
print(b0, b1)    # ≈ 2.2, 0.6
```

2.4 Advantages / Limitations

Advantages	Limitations
Simple, fast, interpretable	Assumes linearity
Works well on linearly related data	Sensitive to outliers
Baseline for regression	Multicollinearity hurts coefficients
	Can't capture complex patterns (use polynomial/regularized/tree models)

Related (extra): Polynomial regression (add x^2, x^3), Ridge (L2) and Lasso (L1) regularization to reduce overfitting.

---

3. Model Evaluation Metrics for Regression

Let y_i = actual, $\hat{y}_i$ = predicted, $\bar{y}$ = mean of actual, n = samples.

3.1 MAE: Mean Absolute Error

MAE = (1/n) Σ $|y_i - \hat{y}_i|$ Average size of errors, in y's units. Robust to outliers; easy to interpret.

3.2 MSE: Mean Squared Error

MSE = (1/n) Σ $(y_i - \hat{y}_i)^2$ Squares errors → penalizes large errors heavily; units are y^2. Differentiable (used as loss).

3.3 RMSE: Root Mean Squared Error

RMSE = $\sqrt{}$MSE Back in y's units; sensitive to outliers. RMSE ≥ MAE always.

3.4 R^2: Coefficient of Determination

$R^2 = 1 - SSres / SStot$, where SSres = $\Sigma(y_i - \hat{y}_i)^2$, SStot = $\Sigma(y_i - \bar{y})^2$

- Fraction of variance in y explained by the model.
- $R^2 = 1$: perfect; $R^2 = 0$: no better than predicting the mean; **negative**: worse than the mean.
- **Adjusted R^2** = $1 - [(1 - R^2)(n - 1)/(n - p - 1)]$ (p = number of features). It doesn't increase just by adding useless features, so it's better for MLR.

3.5 Worked Example

Actual y	Predicted ŷ	Error	|Error|	Error²
3	2.5	0.5	0.5	0.25
5	5.0	0	0	0
7	8.0	−1.0	1.0	1.00
9	8.5	0.5	0.5	0.25

MAE = 2/4 = **0.5**; MSE = 1.5/4 = **0.375**; RMSE = $\sqrt{}0.375 \approx$ **0.612**. $\bar{y} = 6$ → SStot = 9+1+1+9 = 20; SSres = 1.5 → $R^2 = 1 - 1.5/20$ = **0.925**.

3.6 Code

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
y_true = [3, 5, 7, 9]; y_pred = [2.5, 5, 8, 8.5]

mae  = mean_absolute_error(y_true, y_pred)
mse  = mean_squared_error(y_true, y_pred)
rmse = np.sqrt(mse)
r2   = r2_score(y_true, y_pred)
print(f"MAE={mae:.3f} MSE={mse:.3f} RMSE={rmse:.3f} R2={r2:.3f}")

# Adjusted R2
n, p = len(y_true), 2
adj_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)
```

3.7 Which metric?

Metric	Prefer when
MAE	outliers shouldn't dominate; want easy interpretation

RMSE	large errors are especially bad
R^2	explain how much variance is captured; compare models on the same data

---

4. Logistic Regression

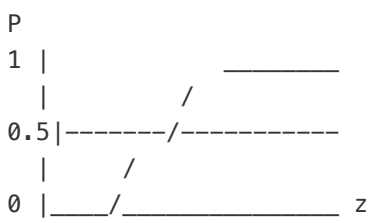
4.1 Why not Linear Regression for classification?

Linear output can be < 0 or > 1 (not a valid probability) and is distorted by outliers. Logistic regression squashes the output into **(0, 1)**.

4.2 The Model

1. Linear part: $\mathbf{z = \beta_0 + \beta_1 x_1 + \ldots + \beta_n x_n}$
2. **Sigmoid (logistic) function: $\sigma(z) = 1 / (1 + e^{-z})$**
3. **$P(y = 1 \mid x) = \sigma(z)$**
4. Prediction: $\hat{y} = 1$ if $P \geq 0.5$ (default threshold) else 0.

Sigmoid properties: $\sigma(0) = 0.5$; $\sigma(+\infty) \to 1$; $\sigma(-\infty) \to 0$; S-shaped curve.



Odds & logit: odds = p/(1–p); **log-odds = ln(p/(1–p)) = z** (linear in features). $\exp(\beta_j)$ = odds ratio for a unit increase in x_j.

Decision boundary: where z = 0 (linear boundary).

4.3 Cost Function: Log Loss (Binary Cross-Entropy)

$J(\beta) = -(1/n)\ \Sigma\ [\ y_i \cdot \log(p_i) + (1 - y_i) \cdot \log(1 - p_i)\ ]$ Convex; minimized by gradient descent (or solvers like lbfgs). Confidently wrong predictions are penalized heavily.

4.4 Worked Example

Model: z = –4 + 1.5 × (study hours). For 3 hours: z = 0.5 → p = $1/(1+e^{-0.5}) \approx$ **0.622** → predict **Pass (1)**. For 2 hours: z = –1 → p ≈ 0.269 → **Fail (0)**.

4.5 Types

- **Binary:** 2 classes.
- **Multinomial:** >2 classes using **softmax** ($\sigma(z_k) = e^{z_k}/\Sigma e^{z_j}$).
- **Ordinal:** ordered classes.
- sklearn handles multi-class automatically (`multi_class="auto"`).

4.6 Regularization

`C` = inverse of regularization strength (small C → strong regularization). `penalty="l2"` (default) or `"l1"`

4.7 Code

```
import pandas as pd, numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s  = scaler.transform(X_test)

clf = LogisticRegression(max_iter=1000, C=1.0)
clf.fit(X_train_s, y_train)

y_pred  = clf.predict(X_test_s)
y_proba = clf.predict_proba(X_test_s)[:, 1]     # probability of class 1

print("Accuracy:", accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))

# Custom threshold (e.g., favour recall)
y_pred_03 = (y_proba >= 0.3).astype(int)

# Coefficients
print(pd.Series(clf.coef_[0], index=X.columns).sort_values().tail(5))
```

4.8 Pros & Cons

Pros	Cons
Simple, fast, gives probabilities	Linear decision boundary only
Interpretable (odds ratios)	Needs feature scaling/regularization for best results
Works as a strong baseline	Struggles with highly non-linear data

---

5. Introduction to Classification Problems

Classification: predict a categorical label from features.

5.1 Types

Type	Description	Example

Binary	2 classes	Spam / Not spam
Multi-class	>2 mutually exclusive classes	Iris species (3), digits 0–9
Multi-label	Multiple labels per instance	A movie tagged Action + Comedy
Imbalanced	One class rare	Fraud (0.1%)

5.2 Process

Collect labeled data → preprocess (encode, scale) → split → train classifier → predict class/probability → evaluate (confusion matrix etc.) → tune.

5.3 Common Classifiers (overview)

Algorithm	Idea
Logistic Regression	Sigmoid on linear score
k-NN	Vote of nearest neighbours
Decision Tree	If-else splits maximizing purity (Gini/Entropy)
Naive Bayes	Bayes' theorem + feature independence
SVM	Maximum-margin hyperplane
Random Forest	Ensemble of trees

5.4 Decision Boundary & Overfitting

A classifier divides feature space into regions per class. Very flexible boundaries fit noise (overfit); overly simple ones underfit.

---

6. k-Nearest Neighbors (k-NN)

6.1 Concept

"Tell me who your neighbours are, and I'll tell you who you are." A new point is assigned the **majority class among its k closest training points**.

- **Lazy learner / instance-based:** no explicit training; stores all data, computation happens at prediction time.
- **Non-parametric:** makes no assumption about data distribution.
- Used for classification (vote) and regression (average of neighbours' values).

6.2 Algorithm

1. Choose **k**.
2. Compute the distance from the new point to every training point.
3. Sort and pick the **k nearest**.
4. **Classification:** majority vote; **Regression:** mean.
5. (Optional) weighted voting, with closer neighbours counting more (`weights="distance"`)

5. (Optional) weighted voting, with closer neighbours counting more (`weights="distance"`).

6.3 Distance Metrics

For points p = (p_1..p_n), q = (q_1..q_n):

- **Euclidean:** d = $\sqrt{\Sigma(p_i - q_i)^2}$ (straight line; default)
- **Manhattan:** d = $\Sigma|p_i - q_i|$ (grid/city block)
- **Minkowski:** d = $(\Sigma|p_i - q_i|^p)^{1/p}$ (p=1 Manhattan, p=2 Euclidean)
- **Hamming:** count of differing positions (categorical)
- **Cosine:** angle-based (text)

6.4 Worked Example

Training data (Height cm, Weight kg → Class): A(170, 65) → M, B(160, 55) → S, C(175, 70) → M, D(155, 50) → S Query Q = (168, 62), k = 3.

- d(Q,A) = $\sqrt{(4+9)}$ = 3.61
- d(Q,B) = $\sqrt{(64+49)}$ = 10.63
- d(Q,C) = $\sqrt{(49+64)}$ = 10.63
- d(Q,D) = $\sqrt{(169+144)}$ = 17.69 3 nearest: A (M), B (S), C (M) → votes M=2, S=1 → **Predict M**.

6.5 Choosing k

Small k (e.g., 1)	Large k
Low bias, high variance	High bias, low variance
Overfits, sensitive to noise	Underfits, blurry boundary

- Use **odd k** for binary classification to avoid ties.
- Rule of thumb: k ≈ $\sqrt{n}$; best chosen by **cross-validation** (elbow in error-vs-k plot).

6.6 Important Practical Points

- **Feature scaling is mandatory** (large-range features dominate distance): use StandardScaler/MinMaxScaler.
- **Curse of dimensionality:** distances become meaningless in many dimensions; use feature selection/PCA.
- Slow for large datasets at prediction time (O(n·d) per query); use KD-tree/Ball-tree.
- Sensitive to irrelevant features and outliers.

6.7 Code

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

iris = load_iris()
X, y = iris.data, iris.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Scale (fit on train only)
sc = StandardScaler()
X_train_s = sc.fit_transform(X_train)
X_test_s = sc.transform(X_test)

# Train
knn = KNeighborsClassifier(n_neighbors=5, metric="euclidean", weights="uniform")
knn.fit(X_train_s, y_train)
y_pred = knn.predict(X_test_s)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=iris.target_names))

# Find the best k using 5-fold cross-validation
ks = range(1, 26)
scores = [cross_val_score(KNeighborsClassifier(n_neighbors=k), X_train_s, y_train, cv=5).
          for k in ks]
best_k = ks[int(np.argmax(scores))]
print("Best k:", best_k)

plt.plot(ks, scores, marker="o")
plt.xlabel("k"); plt.ylabel("CV Accuracy"); plt.title("Choosing k"); plt.show()

# Predict a new sample
new = sc.transform([[5.1, 3.5, 1.4, 0.2]])
print(iris.target_names[knn.predict(new)[0]], knn.predict_proba(new))
```

From scratch (for exams):

```
from collections import Counter
def knn_predict(X_train, y_train, x_new, k=3):
    dists = np.sqrt(((X_train - x_new) ** 2).sum(axis=1))   # Euclidean
    idx = np.argsort(dists)[:k]                              # k nearest
    return Counter(y_train[idx]).most_common(1)[0][0]        # majority vote
```

6.8 Pros & Cons

Pros	Cons
Simple, intuitive	Slow prediction on big data
No training phase	Needs scaling
Handles multi-class naturally	Poor in high dimensions
Non-linear boundaries	Memory heavy; sensitive to noise/imbalance

---

7. Performance Metrics for Classification

7.1 Confusion Matrix

For binary classification (positive = class of interest):

	Predicted Positive	Predicted Negative
Actual Positive	**TP** (True Positive)	**FN** (False Negative, Type II error)
Actual Negative	**FP** (False Positive, Type I error)	**TN** (True Negative)

Example (disease test): TP = sick correctly flagged; FN = sick person missed (dangerous!); FP = healthy person flagged; TN = healthy correctly cleared.

7.2 Metrics

Metric	Formula	Answers
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Overall, how often is the model right?
Precision	TP / (TP + FP)	When it predicts positive, how often is it right?
Recall (Sensitivity, TPR)	TP / (TP + FN)	Of all actual positives, how many did it find?
Specificity (TNR)	TN / (TN + FP)	Of all actual negatives, how many did it clear?
F1-score	2·P·R / (P + R)	Balance between precision and recall (harmonic mean)
FPR	FP / (FP + TN)	False alarm rate

Why harmonic mean? It is dominated by the lower value, so F1 is high only when both precision and recall are high.

7.3 When to prioritize what

Situation	Prioritize
Spam filter (don't lose real mail)	**Precision** (FP costly)
Cancer screening, fraud detection (don't miss cases)	**Recall** (FN costly)
Need balance, imbalanced data	**F1-score**
Balanced classes	Accuracy

⚠ **Accuracy paradox:** with 99% negatives, a model that always predicts "negative" gets 99% accuracy but **0% recall**.

7.4 Worked Example

Out of 100 patients: TP = 40, FP = 10, FN = 20, TN = 30.

- Accuracy = (40+30)/100 = **0.70**
- Precision = 40/(40+10) = **0.80**

- Recall = 40/(40+20) = **0.667**
- Specificity = 30/(30+10) = **0.75**
- F1 = 2(0.8)(0.667)/(0.8+0.667) = **0.727**

7.5 Multi-class Averaging

- **Macro:** simple average of per-class metrics (treats classes equally).
- **Weighted:** average weighted by class support (handles imbalance).
- **Micro:** pool all TP/FP/FN globally.

7.6 Threshold, ROC & AUC (extra)

Changing the probability threshold trades precision vs recall. **ROC curve** plots TPR (recall) vs FPR at all thresholds; **AUC** = area under it (1 = perfect, 0.5 = random).

7.7 Code

```
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             confusion_matrix, classification_report,
                             roc_auc_score, roc_curve, ConfusionMatrixDisplay)
import seaborn as sns, matplotlib.pyplot as plt

y_true = [1,0,1,1,0,1,0,0,1,0]
y_pred = [1,0,1,0,0,1,1,0,1,0]

cm = confusion_matrix(y_true, y_pred)
tn, fp, fn, tp = cm.ravel()
print("TP, FP, FN, TN =", tp, fp, fn, tn)

print("Accuracy :", accuracy_score(y_true, y_pred))
print("Precision:", precision_score(y_true, y_pred))
print("Recall   :", recall_score(y_true, y_pred))
print("F1       :", f1_score(y_true, y_pred))
print(classification_report(y_true, y_pred))

# Heatmap
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=["Pred 0","Pred 1"], yticklabels=["Act 0","Act 1"])
plt.xlabel("Predicted"); plt.ylabel("Actual"); plt.show()

# ROC-AUC (needs probabilities)
# y_proba = model.predict_proba(X_test)[:, 1]
# print(roc_auc_score(y_test, y_proba))
# fpr, tpr, thr = roc_curve(y_test, y_proba); plt.plot(fpr, tpr)

# Multi-class
# precision_score(y_true, y_pred, average="macro")
```

---

8. Unit 2 Revision & Practice

Comparison: Linear vs Logistic Regression

Linear Regression	Logistic Regression

Task	Regression	Classification
Output	Any real number	Probability (0–1)
Function	Straight line	Sigmoid curve
Loss	MSE	Log loss
Metrics	RMSE, R^2	Accuracy, F1, AUC

Quick Revision

- Supervised = labeled data; Regression (continuous) vs Classification (discrete).
- SLR: $\hat{y} = \beta_0 + \beta_1 x$; MLR: many features; fit by least squares/gradient descent.
- Metrics: MAE, MSE, RMSE, R^2 (adjusted R^2 for MLR).
- Logistic regression: sigmoid, threshold 0.5, log loss.
- k-NN: lazy, distance-based, scale features, choose k by CV.
- Classification metrics from the confusion matrix: Accuracy, Precision, Recall, F1.

Short Questions

1. Differentiate regression and classification.
2. Define residual. What does least squares minimize?
3. What does $R^2 = 0.85$ mean?
4. Why is RMSE preferred over MSE for reporting?
5. Why is "regression" in the name logistic regression though it classifies?
6. Why is k-NN called a lazy learner?
7. Why must features be scaled for k-NN?
8. What is the accuracy paradox?
9. Define precision and recall; when is each important?

Long Questions

1. Explain simple and multiple linear regression with assumptions, equations and code.
2. Explain regression evaluation metrics with formulas and an example.
3. Explain logistic regression (sigmoid, cost function, decision boundary) with code.
4. Explain the k-NN algorithm with a numerical example; discuss the choice of k and distance metrics.
5. Explain the confusion matrix and the classification metrics; compute them for a given matrix.

Numerical Practice

1. Fit a line for x = [1,2,3,4], y = [3,5,7,9]. *Ans:* $\beta_1 = 2$, $\beta_0 = 1 \rightarrow \hat{y} = 1 + 2x$ (perfect fit, $R^2 = 1$).
2. Confusion matrix TP = 50, FP = 5, FN = 10, TN = 35. *Ans:* Accuracy = 0.85, Precision = 0.909, Recall = 0.833, F1 = 0.870.
3. Logistic model $z = -6 + 2x$, find P(y=1) for x = 3. *Ans:* $z = 0 \rightarrow P = 0.5$ (on the decision boundary).
4. k-NN: find the class of the query (2,2) given A(1,1)→0, B(2,3)→1, C(4,4)→1, k=3. *Ans:* d = 1.41, 1,

2.83 → neighbours A, B, C → votes 1:2, 0:1 → class **1**.

Common Mistakes to Avoid

- Fitting the scaler/imputer on the whole data (leakage), so fit on train only.
- Using accuracy on imbalanced data.
- Passing a 1-D array as X to sklearn (`X` must be 2-D).
- Forgetting to scale features for k-NN/logistic regression.
- Interpreting correlation or coefficients as causation.